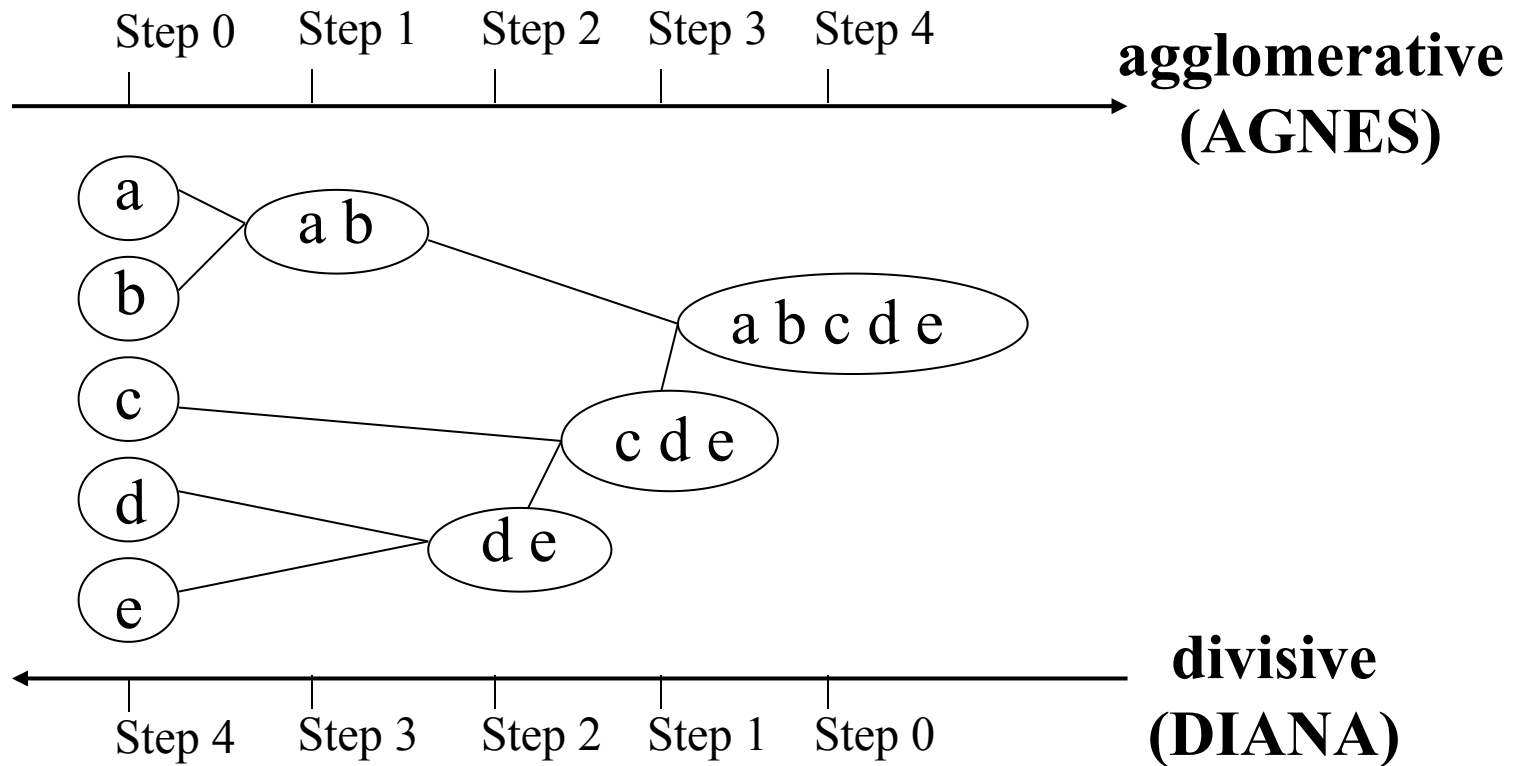


基于层次的聚类

主要思想:

把数据对象排列成一个聚类树，在需要的层次上对其进行切割，相关联的部分构成一个cluster。基于层次的聚类方法有两种类型：

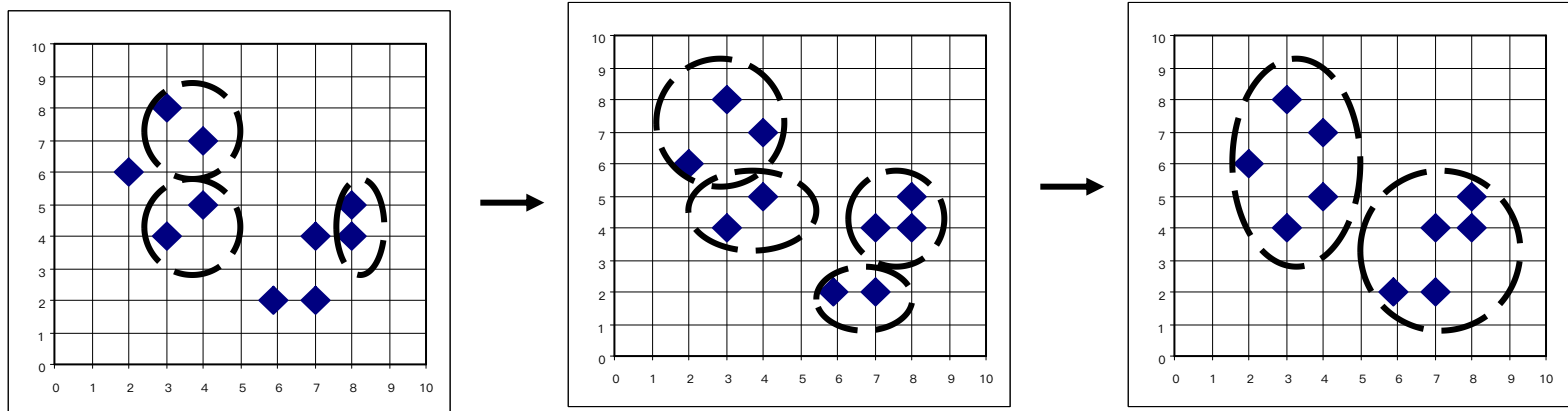
- (1) 聚合层次聚类。最初每个对象是一个cluster，然后根据它们之间的相似性，对这些原子的cluster进行合并。大多数层次方法属于这一类，它们的主要区别是cluster之间的相似性的定义不同。
- (2) 划分层次聚类，它与上面的过程正好相反。



-
- 用户可以指定算法终止的条件，例如，聚类的个数或每个**cluster**的半径低于某个阈值。
 - 弱点在于合并或分裂点的选取问题，因为一组对象一旦合并或分裂，就不能有**undo**的操作
 - 时间复杂度为 $O(N^2)$ ，对于处理大数据量有性能问题。

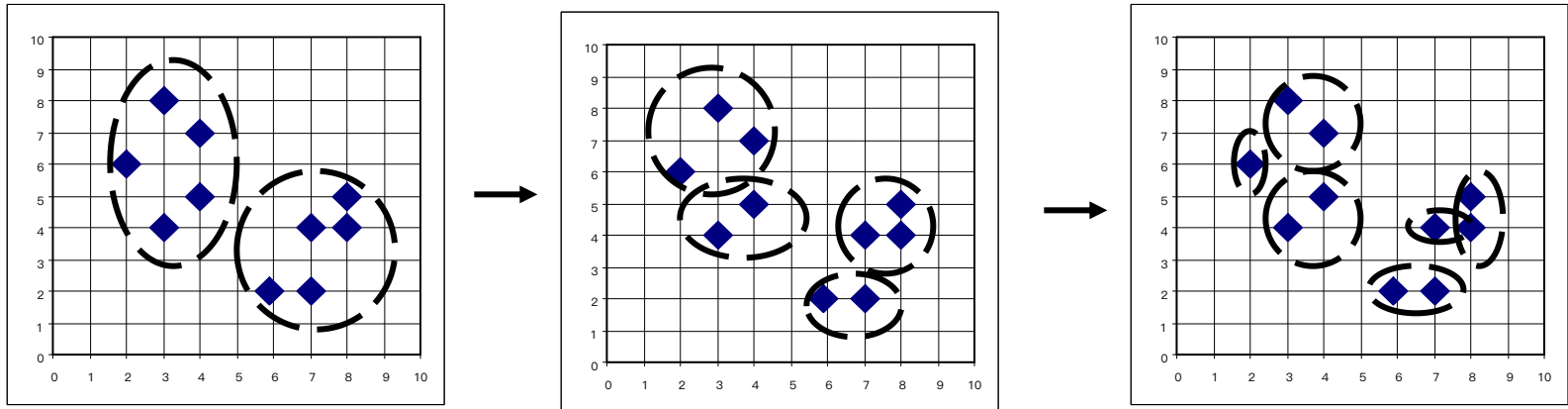
AGNES (Agglomerative Nesting)

- Introduced in Kaufmann and Rousseeuw (1990)
- Implemented in statistical analysis packages, e.g., Splus
- Use the Single-Link method and the dissimilarity matrix.
- Merge nodes that have the least dissimilarity
- Go on in a non-descending fashion
- Eventually all nodes belong to the same cluster



DIANA (Divisive Analysis)

- Introduced in Kaufmann and Rousseeuw (1990)
- Implemented in statistical analysis packages, e.g., Splus
- Inverse order of AGNES
- Eventually each node forms a cluster on its own



基于层次的聚类-族间距离

- 聚合层次聚类和划分层次聚类都是通过计算族间的距离来进行族的合并或者划分的。
- 设 $d(p, q)$ 为两个数据对象 p 和 q 之间的距离， m_i 和 m_j 分别为族 C_i 和 C_j 的中心， n_i 和 n_j 分别代表族 C_i 和 C_j 中数据对象的个数，则族 C_i 和 C_j 之间的距离定义如下：

- 最小距离：
$$d_{\min}(C_i, C_j) = \min_{p \in C_i, q \in C_j} d(p, q)$$

- 最大距离：
$$d_{\max}(C_i, C_j) = \max_{p \in C_i, q \in C_j} d(p, q)$$

基于层次的聚类-族间距离

- 中心点距离: $d_{mean}(C_i, C_j) = d(m_i, m_j)$
- 平均距离: $d_{avg}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p \in C_i} \sum_{q \in C_j} d(p, q)$

基于层次的聚类-步骤

- 以聚合层次聚类为例，根据族之间的距离对相似的族进行逐步合并的过程如下：
 - [1] 每个数据对象自己构成一个原子族。
 - [2] 计算所有原子族之间的两两距离。
 - [3] 将距离最近的两个族进行合并，族的个数减1。
 - [4] 如果满足终止条件，则算法结束。否则计算新生成的族和其他族之间的距离并转步骤[3]。

基于层次的聚类-步骤

- 算法终止的条件可以由用户指定。例如，可以指定当族的个数达到某个阈值或者每个族的半径低于某个阈值时算法停止。族的半径通常定义为：

$$R = \frac{1}{n} \sum_{i=1}^n d(p_i - m)$$

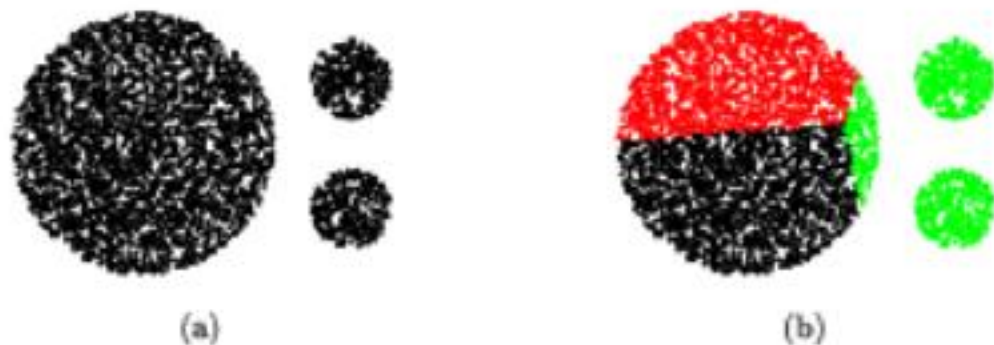
其中， p_i 为族中的每一个数据对象， m 代表族的中心， n 代表族中的数据对象个数。

基于层次的聚类

- 可以看出，聚合层次算法的核心是计算两个族之间的距离并将距离最近的两个族进行合并。
- 如果每一个族的内部都比较紧凑而且族和族之间分离地很好时，采用不同的距离定义所产生的聚类结果差别不大。
- 但是，如果族和族之间分离得不好（哪怕只是存在一些异常点），或者族的形状不是球形，或者族的大小不均匀时，根据不同的距离定义进行聚类的结果将差别很大。

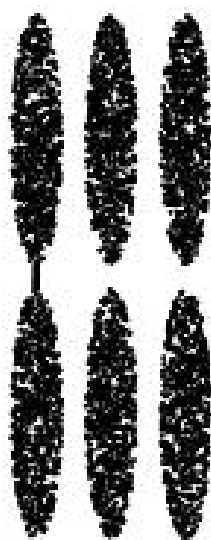
基于层次的聚类

- 例如，在如下 (a) 数据集上应用 $d_{\max}$, d_{avg} , d_{mean} 距离定义时，将产生图 (b) 所示的族结构。

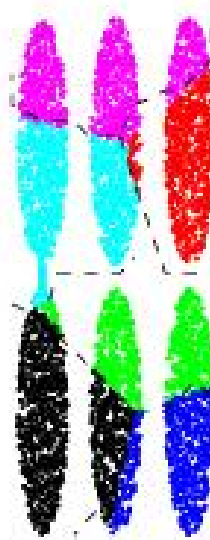


基于层次的聚类

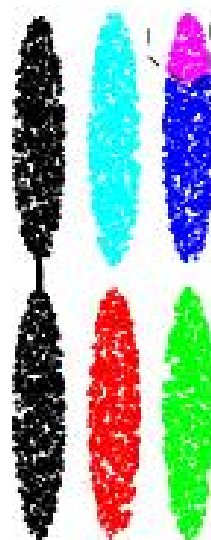
- 类似地，在图 (a) 所示的数据集上应用 d_{mean} 距离定义时，将产生图 (b) 所示的族结构。其中的单个细长的族被切分，而属于邻近的多个细长族的不同部分又被合并。另一方面，当在图 (a) 所示的数据集上应用 d_{min} 距离定义时，将产生图 (c) 所示的族结构。两个细长族被连接了起来，产生了“链条”现象。



(a)



(b)



(c)

基于层次的聚类

- 可以看出， d_{mean} 和 d_{min} 距离定义都不太适合非球形或大小不均匀的族结构。采用 d_{mean} 作为距离定义的缺点是它只取一个点（即族的中心点）作为整个族的代表点。对于一个特别大的族或者形状不规则的族，在进行聚合层次聚类时，它下一层的孩子族的中心点可能离得很远，无法合并成一个族，导致本该属于同一个族的点被分离到不同的族中。
- 与之相反，当采用 d_{min} 作为距离定义时，又走向了另一个极端，考虑了族中所有的点，即将族中的每一个点都作为代表点对待，该方法的缺点是容易受异常点的干扰，数据点位置的微小变化都可能导致聚类结构的变化。

Recent Hierarchical Clustering Methods

- Major weakness of agglomerative clustering methods
 - do not scale well: time complexity of at least $O(n^2)$, where n is the number of total objects
 - can never undo what was done previously
- Integration of hierarchical with distance-based clustering
 - BIRCH (1996): uses CF-tree and incrementally adjusts the quality of sub-clusters
 - CURE (1998) : Clustering Using Representatives
 - ROCK (1999): clustering categorical data by neighbor and link analysis
 - CHAMELEON (1999): hierarchical clustering using dynamic modeling

BIRCH (1996)

- 当数据量很大时，聚合层次聚类算法因为其非线性的时间复杂度和巨大的I/O开销可能会崩溃。为此人们提出了能够适用于大数据量聚类的**BIRCH**算法。
- 该方法首先针对数据集进行预聚类micro-cluster），构建一个初始的内存**CF-Tree**，其保留了数据的内在的聚类特征。
- 然后采用基于 d_{mean} 的聚合层次聚类方法对所有的微族进行聚类（相对于原始数据，这时的数据集已经很小了）。

BIRCH (1996)

- Birch: Balanced Iterative Reducing and Clustering using Hierarchies (Zhang, Ramakrishnan & Livny, SIGMOD'96)
- Incrementally construct a CF (Clustering Feature) tree, a hierarchical data structure for multiphase clustering
 - Phase 1: scan DB to build an initial in-memory CF tree (a multi-level compression of the data that tries to preserve the inherent clustering structure of the data)
 - Phase 2: use an arbitrary clustering algorithm to cluster the leaf nodes of the CF-tree
- *Scales linearly*: finds a good clustering with a single scan and improves the quality with a few additional scans
- *Weakness*: handles only numeric data, and sensitive to the order of the data record.

Similarity Metric(1)

Given a cluster of instances $\{\vec{X}_i\}$, we define:

Centroid: $\vec{X}_0 = \frac{\sum_{i=1}^N \vec{X}_i}{N}$

Radius: average distance from member points to centroid

$$R = \left(\frac{\sum_{i=1}^N (\vec{X}_i - \vec{X}_0)^2}{N} \right)^{\frac{1}{2}}$$

Diameter: average pair-wise distance within a cluster

$$D = \left(\frac{\sum_{i=1}^N \sum_{j=1}^N (\vec{X}_i - \vec{X}_j)^2}{N(N-1)} \right)^{\frac{1}{2}}$$

Similarity Metric(2)

centroid Euclidean distance:

$$D0 = ((\vec{X}0_1 - \vec{X}0_2)^2)^{\frac{1}{2}}$$

centroid Manhattan distance:

$$D1 = |\vec{X}0_1 - \vec{X}0_2| = \sum_{i=1}^d |\vec{X}0_1^{(i)} - \vec{X}0_2^{(i)}|$$

average inter-cluster:

$$D2 = \left(\frac{\sum_{i=1}^{N_1} \sum_{j=N_1+1}^{N_1+N_2} (\vec{X}_i - \vec{X}_j)^2}{N_1 N_2} \right)^{\frac{1}{2}}$$

average intra-cluster:

$$D3 = \left(\frac{\sum_{i=1}^{N_1+N_2} \sum_{j=1}^{N_1+N_2} (\vec{X}_i - \vec{X}_j)^2}{(N_1 + N_2)(N_1 + N_2 - 1)} \right)^{\frac{1}{2}}$$

variance increase:

$$D4 = \left(\sum_{k=1}^{N_1+N_2} \left(\vec{X}_k - \frac{\sum_{l=1}^{N_1+N_2} \vec{X}_l}{N_1+N_2} \right)^2 - \sum_{i=1}^{N_1} \left(\vec{X}_i - \frac{\sum_{l=1}^{N_1} \vec{X}_l}{N_1} \right)^2 - \sum_{j=N_1+1}^{N_1+N_2} \left(\vec{X}_j - \frac{\sum_{l=N_1+1}^{N_1+N_2} \vec{X}_l}{N_2} \right)^2 \right)^{\frac{1}{2}}$$

Clustering Feature

- *The Birch algorithm builds a dendrogram called clustering feature tree (CF tree) while scanning the data set.*

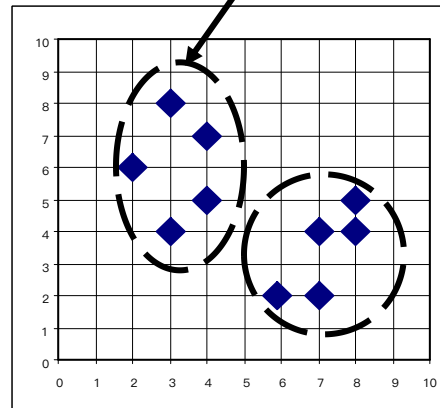
Clustering Feature Vector in BIRCH

Clustering Feature: $CF = (N, LS, SS)$

N : **Number of data points**

$$LS: \sum_{i=1}^N \vec{X}_i$$

$$SS: \sum_{i=1}^N \vec{X}_i^2$$



$$CF = (5, (16,30), (54,190))$$

(3,4)

(2,6)

(4,5)

(4,7)

(3,8)

CF-Tree in BIRCH

- Clustering feature:
 - summary of the statistics for a given subcluster: the 0-th, 1st and 2nd moments of the subcluster from the statistical point of view.
 - registers crucial measurements for computing cluster and utilizes storage efficiently
- A CF tree is a height-balanced tree that stores the clustering features for a hierarchical clustering
 - A nonleaf node in a tree has descendants or “children”
 - The nonleaf nodes store sums of the CFs of their children
- A CF tree has two parameters
 - Branching factor: specify the maximum number of children.
 - threshold: max diameter of sub-clusters stored at the leaf nodes

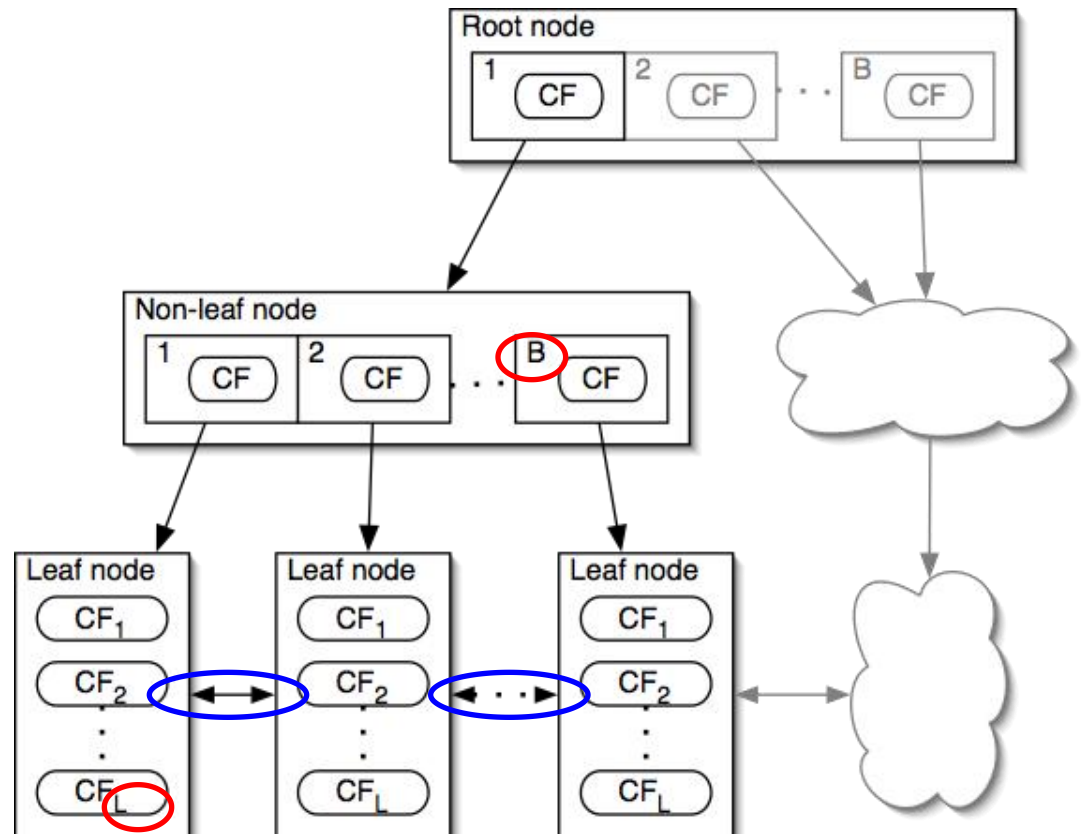
Properties of Clustering Feature

- CF entry is more compact
 - *Stores significantly less than all of the data points in the sub-cluster*
- A CF entry has sufficient information to calculate D0-D4
- Additivity theorem allows us to merge sub-clusters incrementally & consistently

$$\mathbf{CF}_1 + \mathbf{CF}_2 = (N_1 + N_2, \vec{LS}_1 + \vec{LS}_2, SS_1 + SS_2)$$

CF-Tree

- Each non-leaf node has at most B entries
- Each leaf node has at most L CF entries, each of which satisfies threshold T
- Node size is determined by dimensionality of data space and input parameter P (page size)



CF-Tree Insertion

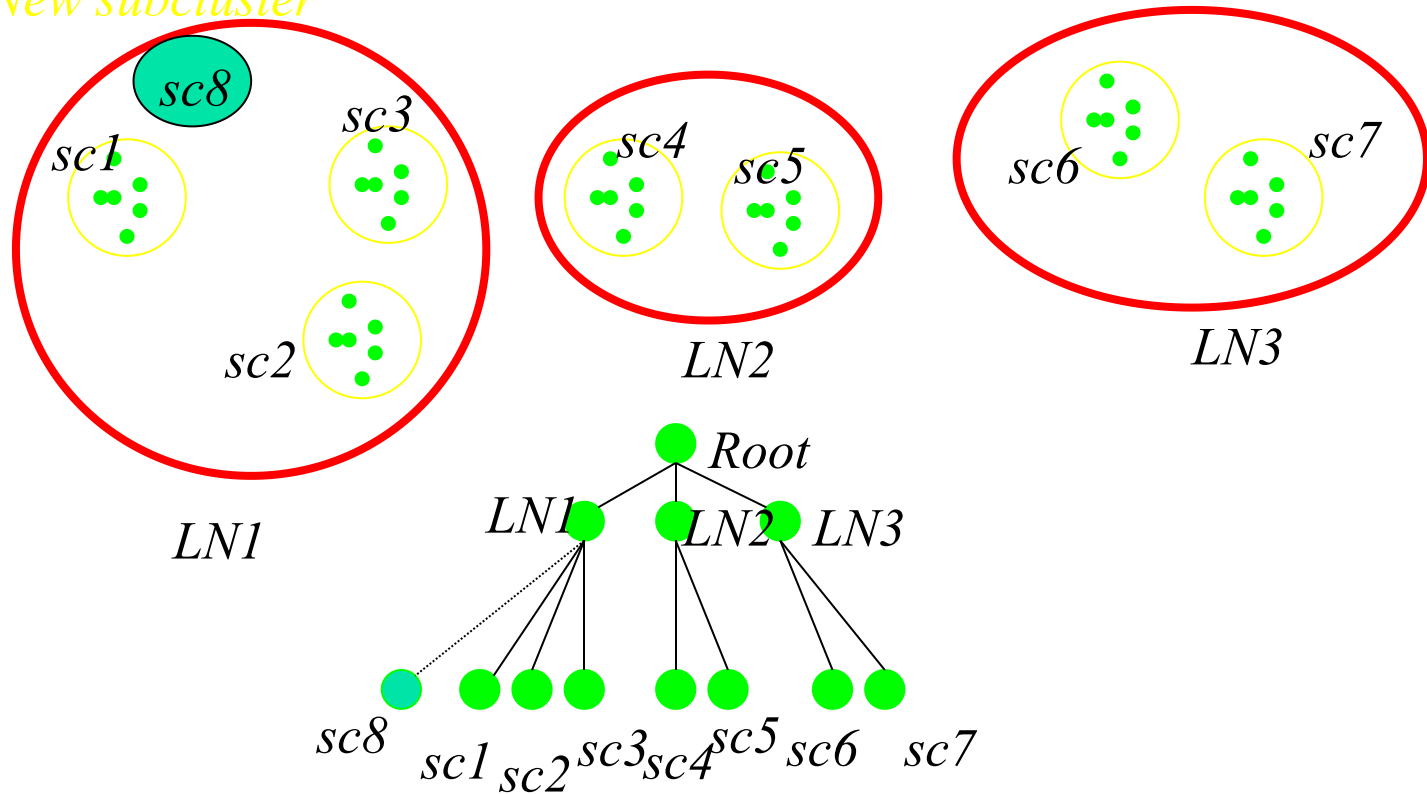
- Recurse down from root, find the appropriate leaf
 - *Follow the "closest"-CF path, w.r.t. D0 / ... / D4*
- Modify the leaf
 - *If the closest-CF leaf cannot absorb, make a new CF entry. If there is no room for new leaf, split the parent node*
- Traverse back
 - *Updating CFs on the path or splitting nodes*

CF-Tree Rebuilding

- If we run out of space, increase threshold T
 - *By increasing the threshold, CFs absorb more data*
- Rebuilding "pushes" CFs over
 - *The larger T allows different CFs to group together*
- Reducibility theorem
 - *Increasing T will result in a CF-tree smaller than the original*
 - *Rebuilding needs at most h extra pages of memory*

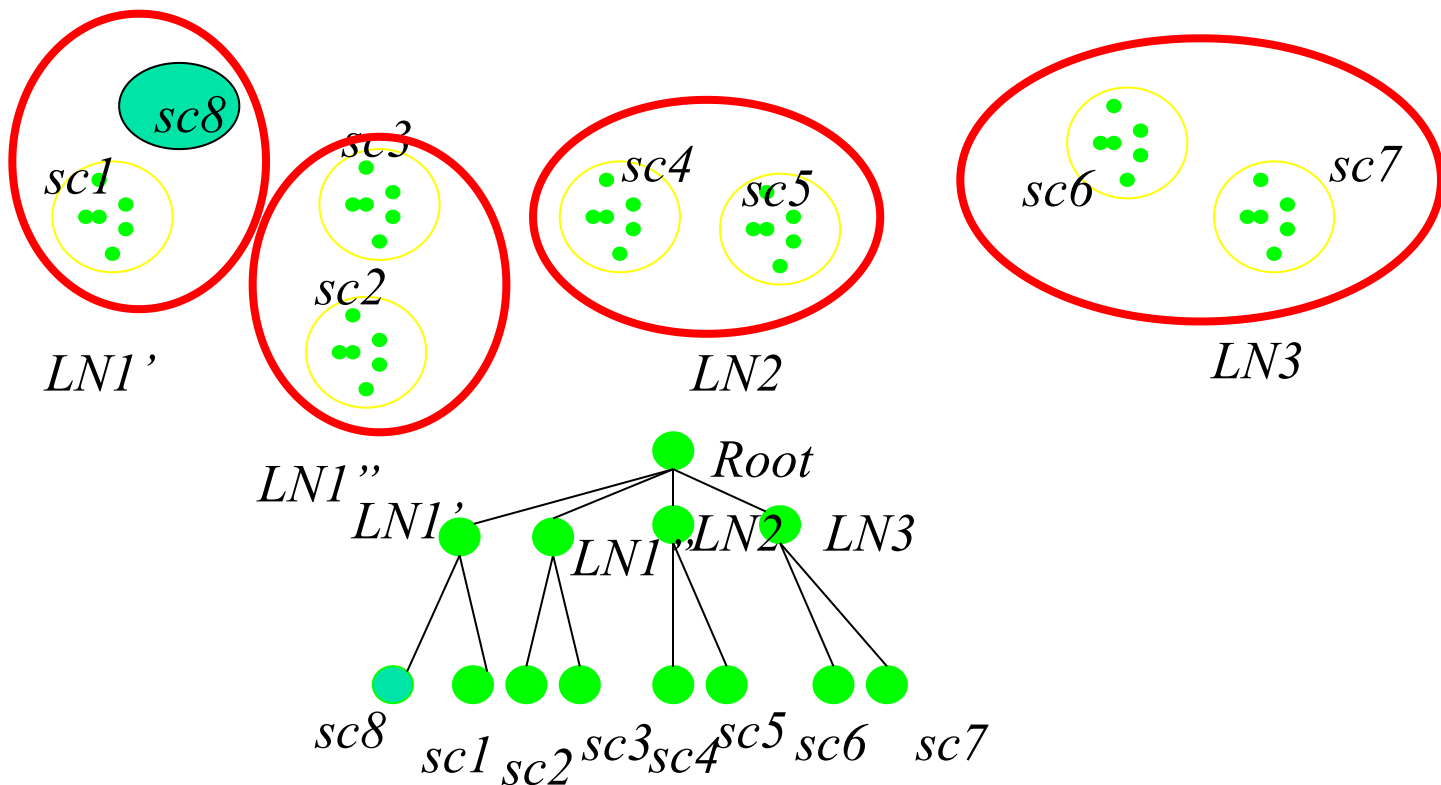
Example of BIRCH

New subcluster

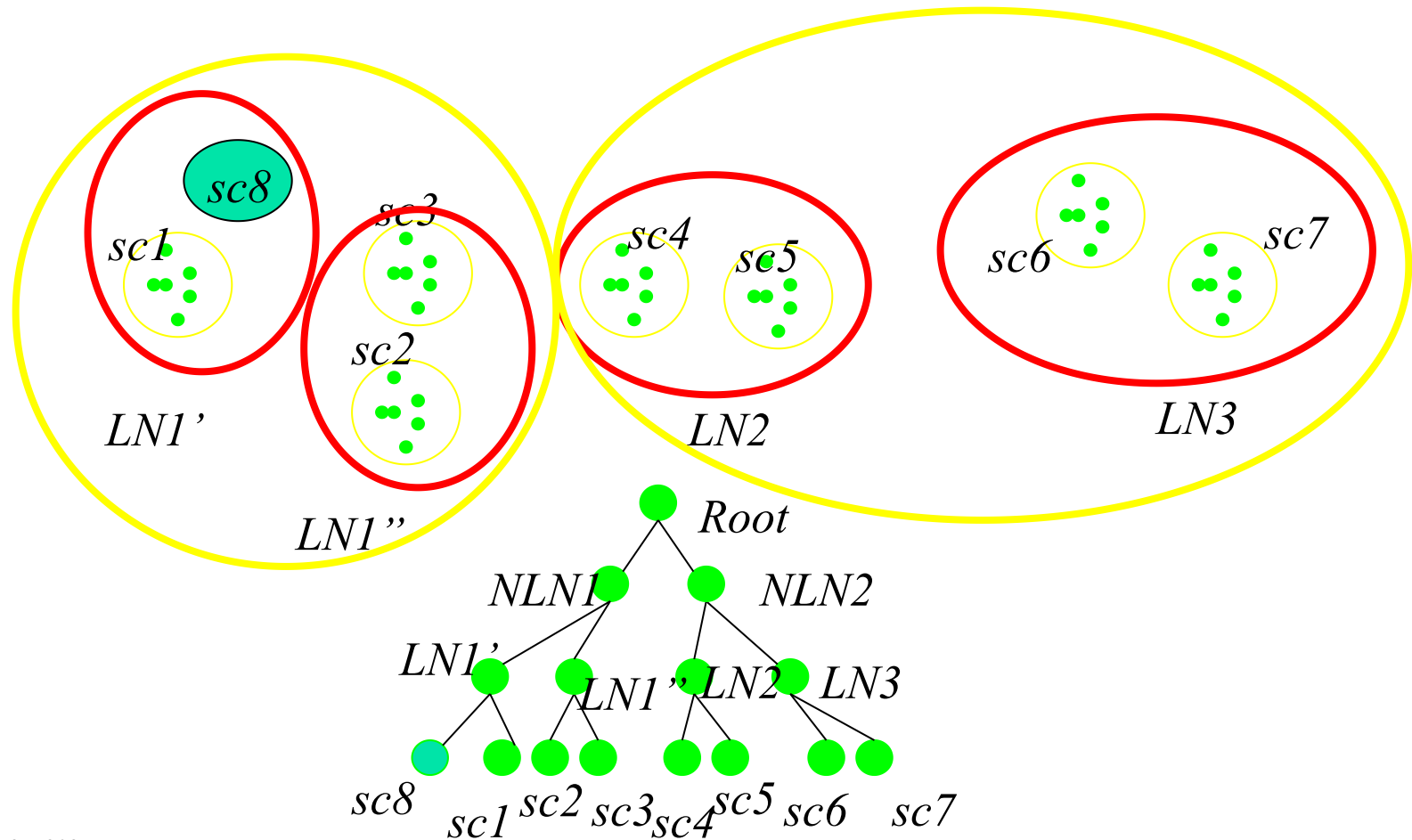


Insertion Operation in BIRCH

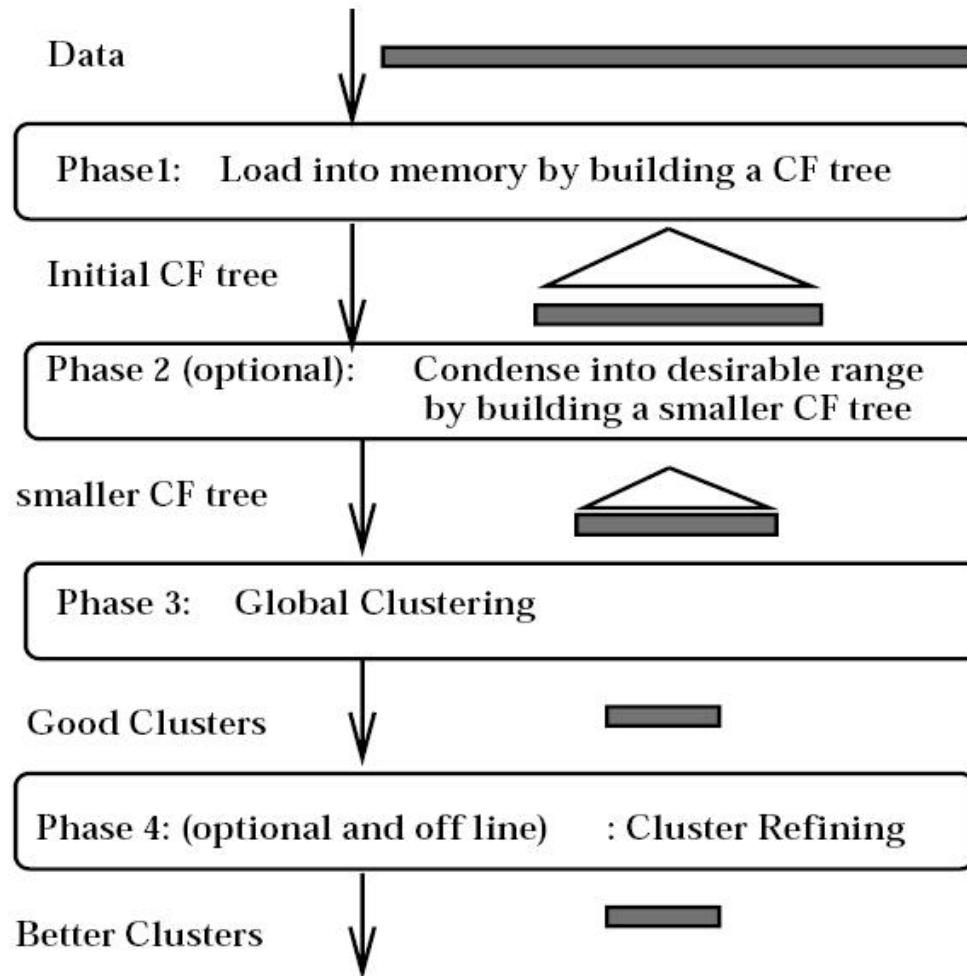
If the branching factor of a leaf node can not exceed 3, then LN1 is split.

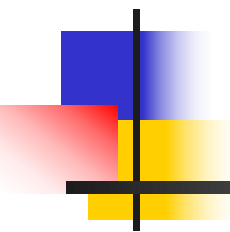


If the branching factor of a non-leaf node can not exceed 3, then the root is split and the height of the CF Tree increases by one.



BIRCH Overview

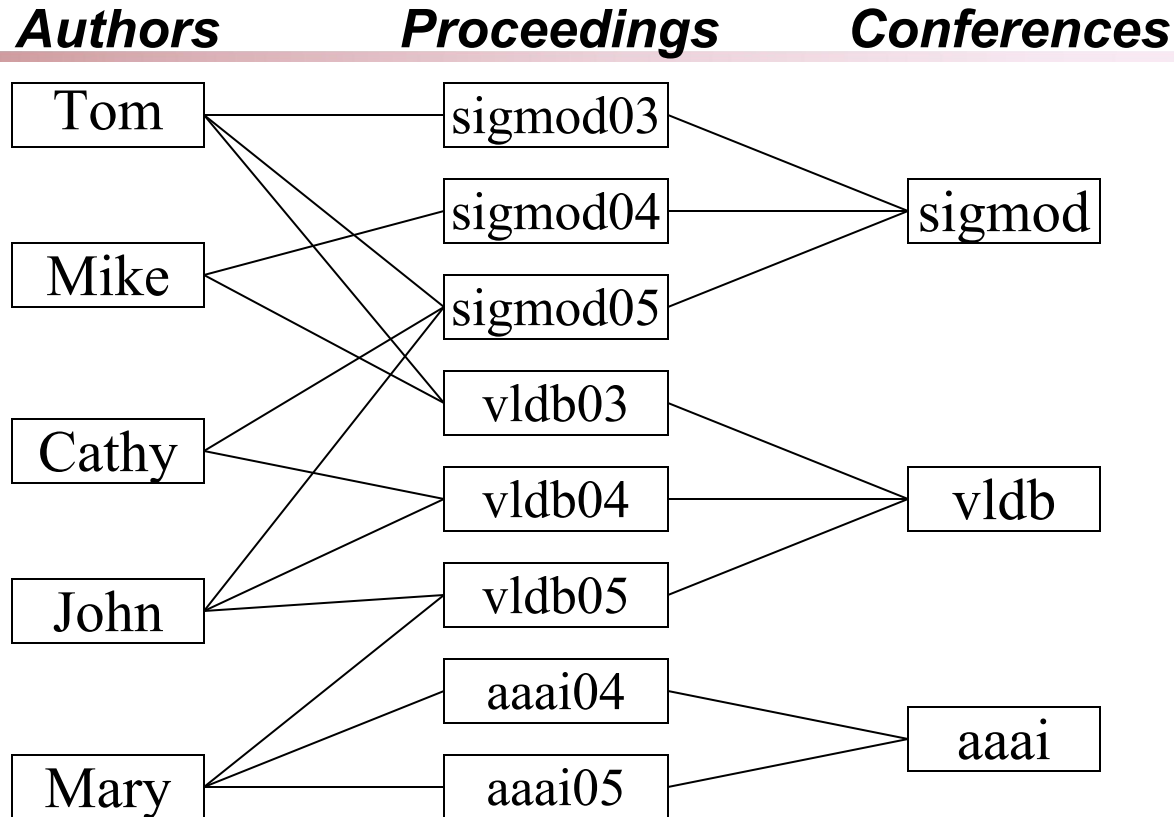




LinkClus: Efficient Clustering via Heterogeneous Semantic Links

Xiaoxin Yin, Jiawei Han
Univ. of Illinois at Urbana-Champaign
Philip S. Yu
IBM T.J. Watson Research Center

A Motivating Example



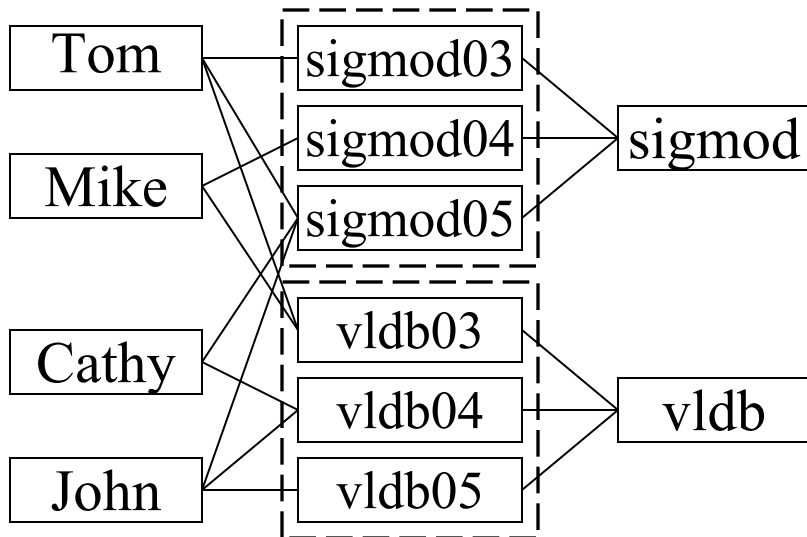
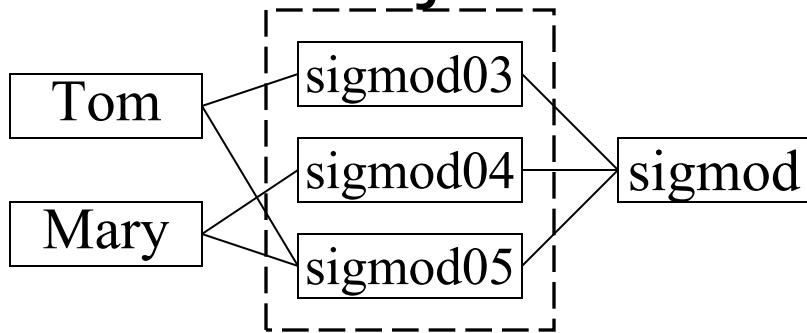
Questions:

Q1: How to cluster each type of objects?

Q2: How to define similarity between each type of objects?

Link-based Similarities

- Two objects are similar if they are linked with similar objects



Jeh & Widom, 2002 - *SimRank*

The similarity between two objects x and y is defined as the average similarity between objects linked with x and those with y .

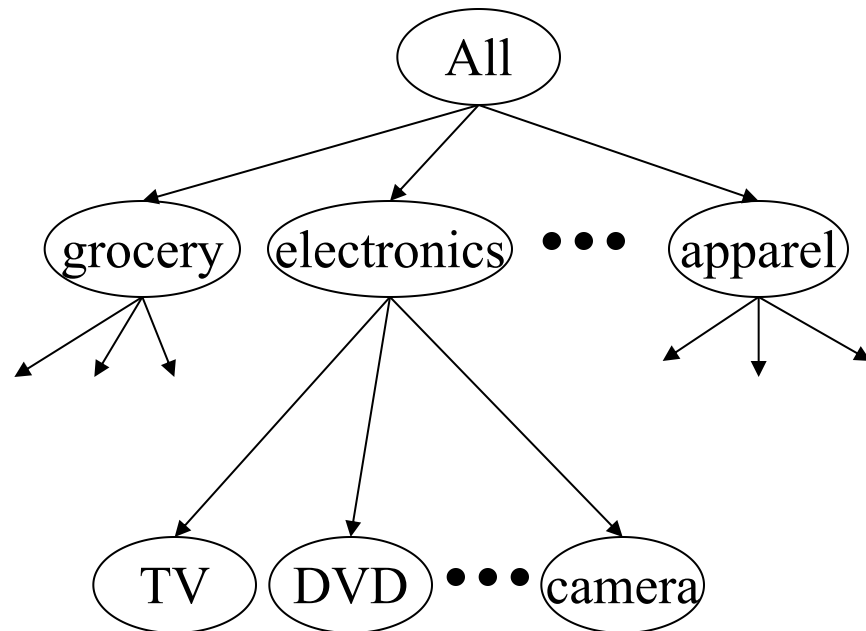
Very expensive to compute:

For a dataset of N objects and M links, it takes $O(N^2)$ space and $O(M^2)$ time to compute all similarities.

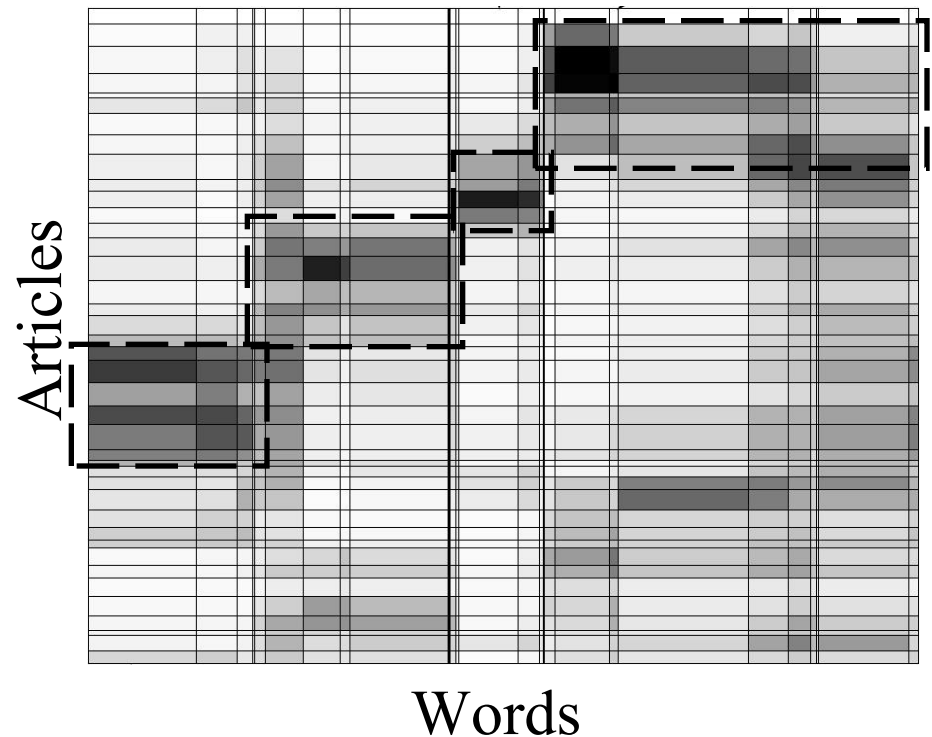
Observation 1: Hierarchical Structures

- Hierarchical structures often exist naturally among objects (e.g., taxonomy of animals)

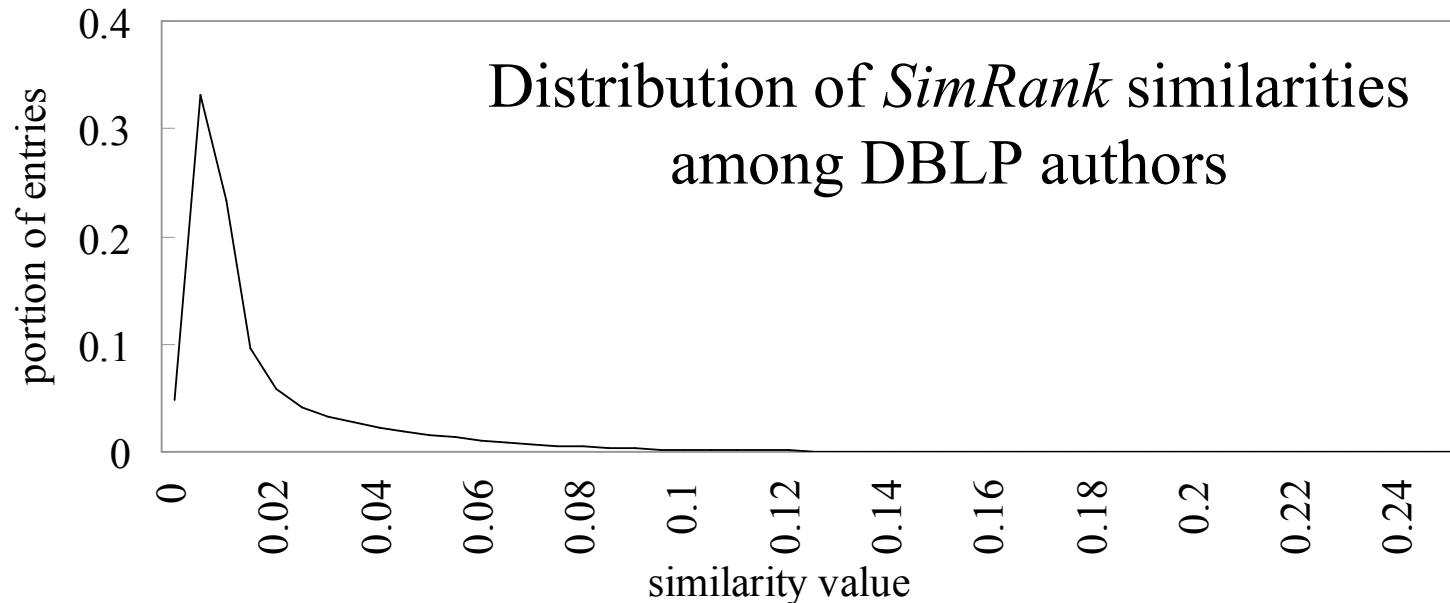
A hierarchical structure of products in Walmart



Relationships between articles and words (Chakrabarti, Papadimitriou, Modha, Faloutsos, 2004)

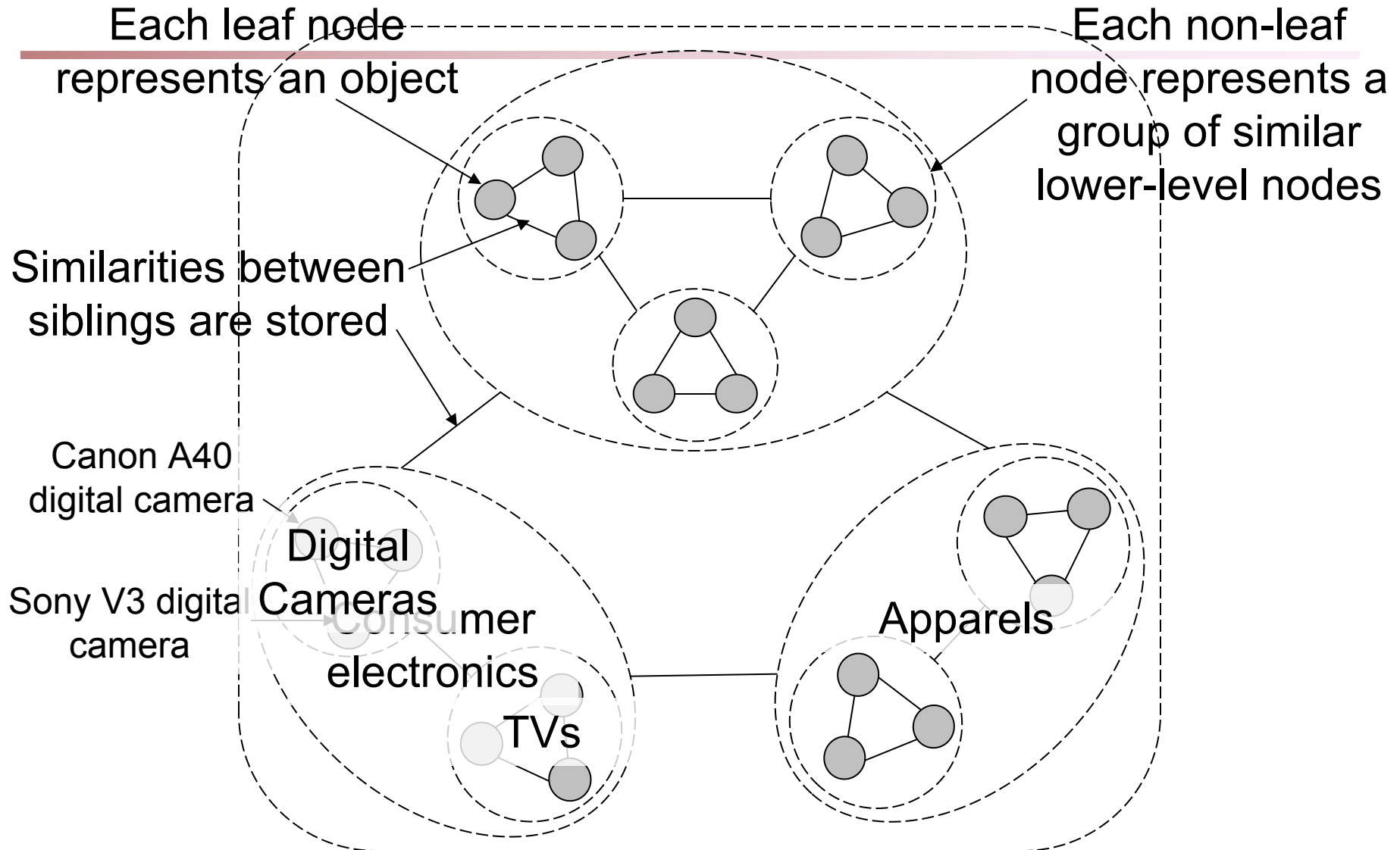


Observation 2: Distribution of Similarity



- Power law distribution exists in similarities
 - 56% of similarity entries are in $[0.005, 0.015]$
 - 1.4% of similarity entries are larger than 0.1
 - Our goal: Design a data structure that stores the significant similarities and compresses insignificant ones

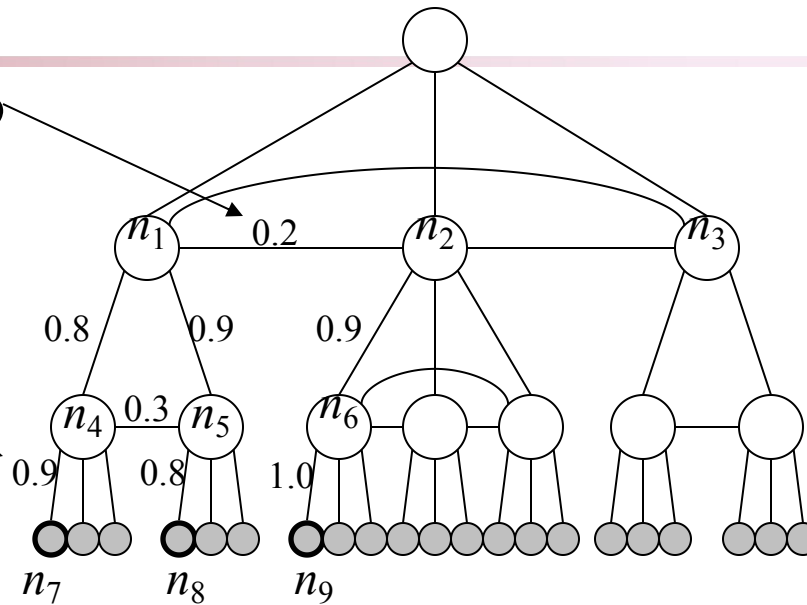
Our Data Structure: SimTree



Similarity Defined by A SimTree

Similarity between two sibling nodes n_1 and n_2

Adjustment ratio for node n_7



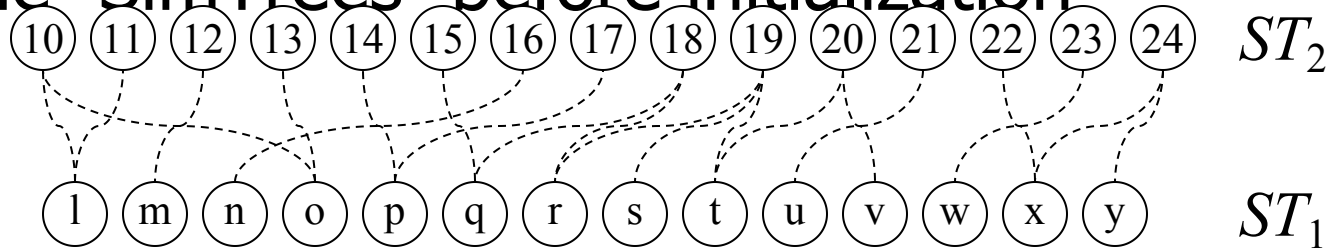
- $sim_p(n_7, n_8) = s(n_7, n_4) \times s(n_4, n_5) \times s(n_5, n_8)$
 - Path-based node similarity
- Similarity between two nodes is the average similarity between objects linked with them in other SimTrees
- Adjustment ratio for $x = \frac{\text{Average similarity between } x \text{ and all other nodes}}{\text{Average similarity between } x\text{'s parent and all other nodes}}$

Overview of LinkClus

- Initialize a SimTree for objects of each type
- Repeat
 - For each SimTree, update the similarities between its nodes using similarities in other SimTrees
 - Similarity between two nodes x and y is the average similarity between objects linked with them
 - Adjust the structure of each SimTree
 - Assign each node to the parent node that it is most similar to

Initialization of SimTrees

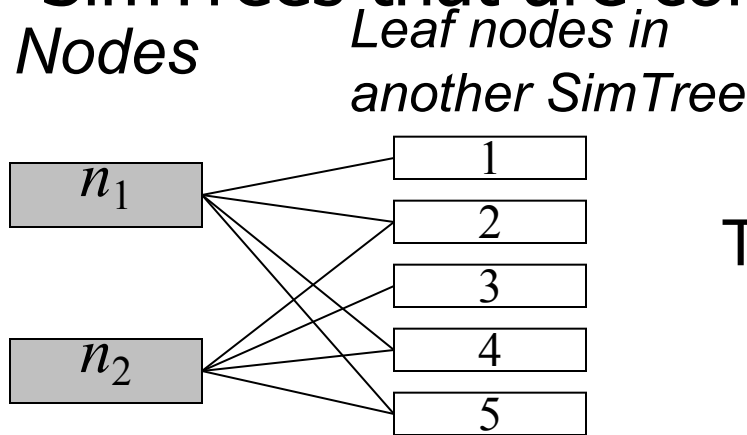
- The “SimTrees” before initialization



- Each leaf nodes have similarity 1 to itself and 0 to others
- Initializing a SimTree
 - Repeatedly find groups of tightly related nodes, which are merged into a higher-level node

(continued)

- Tightness of a group of nodes
 - For a group of nodes $\{n_1, \dots, n_k\}$, its tightness is defined as the number of leaf nodes in other SimTrees that are connected to all of $\{n_1, \dots, n_k\}$



The tightness of $\{n_1, n_2\}$ is 3

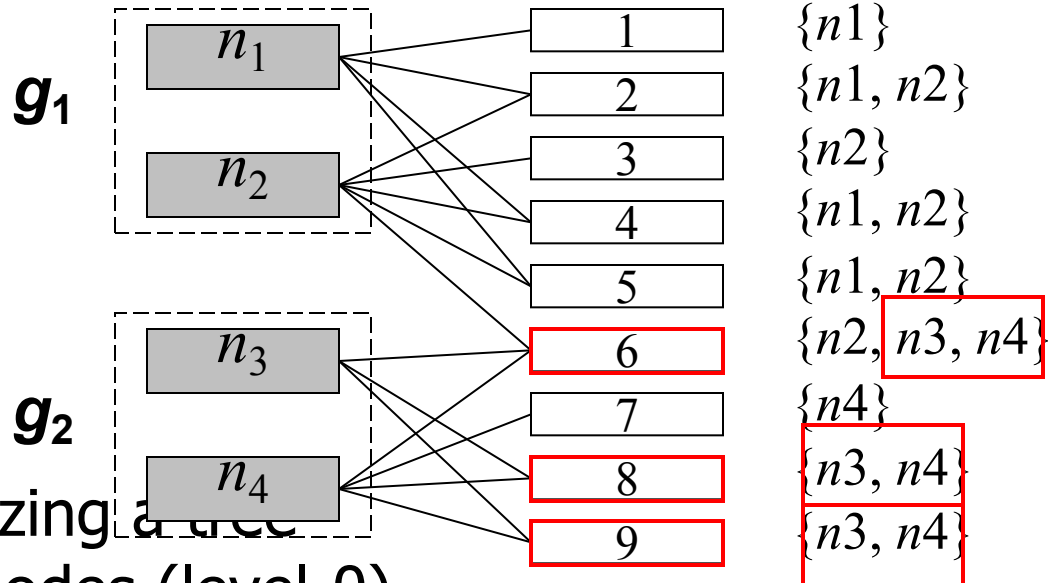
(continued)

- Finding tight groups $\longrightarrow$ Frequent pattern mining

Reduced to

Transactions

The tightness of a group of nodes is the support of a frequent pattern



- Procedure of initializing a tree
 - Start from leaf nodes (level-0)
 - At each level l , find non-overlapping groups of similar nodes with frequent pattern mining